

Reviewer Pack — Schur-Weyl Multiplicity Gap in Permanent vs Determinant Deco...

Entry 60113b1acde4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 21:47:22 UTC

Schur-Weyl Multiplicity Gap in Permanent vs Determinant Decompositions

Entry ID: 60113b1acde4 Verdict: INCONCLUSIVE Recorded: 2026-05-11 21:47:22 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality Field B (complexity object): Boolean Circuits

Statement:

For every CNF formula Φ on n variables with m clauses, the number of irreducible $GL(n)$ -representations in the decomposition of the permanent tensor P_n minus the determinant tensor D_m satisfies $|\mu(P_n) - \mu(D_m)| \geq \Omega(n^{\{1.5\}})$ when $m < n^{\{1.5\}}$.

Rationale (proposer’s reasoning):

Schur-Weyl duality decomposes tensor spaces into symmetric/antisymmetric components, revealing algebraic structure. Permanent and determinant tensors exhibit distinct multiplicity patterns under this decomposition, which may correlate with computational hardness via representation-theoretic obstructions.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2d20f5ef4ae76ca3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random 3-CNF formula with n variables and m clauses
    n = random.randint(5, 40)
    m = random.randint(1, min(n**2, 2*n))
    cnf = []
    for _ in range(m):
        clause = [random.choice([-1, 1]) * (i + 1) for i in random.sample(range(n), 3)]
        cnf.append(clause)

    # Construct the permanent tensor P_n
    P_n = [[1] * n for _ in range(n)]
    for clause in cnf:
        for lit in clause:
            if lit > 0:
                P_n[lit - 1][lit - 1] += 1
            else:
                P_n[-lit - 1][-lit - 1] += 1

    # Construct the determinant tensor D_m
    D_m = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i != j:
                D_m[i][j] = (-1)**(i + j) * (n - 2)

    # Compute the Schur-Weyl decompositions via Young tableau counting
    def hook_length_formula(shape):
        n = len(shape)
        total = 1
        for i in range(n):
            for j in range(len(shape[i])):
                total *= (shape[i][j] + i + j + 1) / (i + j + 1)
        return total

    def dominant_irreducible_multiplicity(tensor, shape):
```

```

n = len(tensor)
m = len(shape)
if n < m:
    return 0
mult = 1
for i in range(m):
    for j in range(len(shape[i])):
        mult *= tensor[i][j] + i + j + 1
        mult //= (i + j + 1)
    return mult

mu_P_n = dominant_irreducible_multiplicity(P_n, [(n - m) // 2] * m)
mu_D_m = dominant_irreducible_multiplicity(D_m, [m // 2] * m)

# Measure the difference in dominant irreducible multiplicities
diff = abs(mu_P_n - mu_D_m)

# Verify the  $\Omega(n^{1.5})$  gap for  $m < n^{1.5}$ 
conjecture_holds = diff >= n**1.5

return {
    "metric_name": "Multiplicity Gap",
    "metric_value": diff,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"m={m}, n={n}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_diff = sum(r["metric_value"] for r in results) / len(results)
    std_diff = math.sqrt(sum((r["metric_value"] - mean_diff)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_diff} std={std_diff} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_diff} std={std_diff} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'Multiplicity Gap', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True}
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3e3a40a9.py", line 89, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3e3a40a9.py", line 66, in run_trial
    mu_P_n = dominant_irreducible_multiplicity(P_n, [(n - m) // 2] * m)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3e3a40a9.py", line 61, in dominant_irreducible_multiplicity
    for j in range(len(shape[i])):
                   ^^^^^^^^^^^^^
TypeError: object of type 'int' has no len()
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with `TypeError` before producing valid results, preventing verification of the counterexample. | next: Fix the `TypeError` in `dominant_irreducible_multiplicity` by ensuring `shape` is a sequence (e.g., list) before `len()` is called

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109985
2	propose	ollama_remote	qwen3:8b	0	0	34823
3	preregistration	ollama_remote	qwen3:8b	0	0	24176
4	novelty	ollama_remote	qwen3:8b	0	0	20726
5	novelty	ollama_remote	qwen3:8b	0	0	14577
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16285
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10344
8	judge	ollama_remote	qwen3:8b	0	0	16395

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 247311 ms total latency. Provider mix: {‘ollama_remote’: 8}

(full prompt+response transcripts available in research/audit/60113b1acde4.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/60113b1acde4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/60113b1acde4.tar.gz` (if generated)